

Threat Analysis Report

รายงานวิเคราะห์ช่องโหว่ PHP ระดับวิกฤต “PHP2Shell” จำนวน 2 รายการ

CVE-2026-109618

CVE-2026-109619

22/10/2026

TLP:CLEAR

สำนักงานคณะกรรมการการรักษาความมั่นคงปลอดภัยไซเบอร์แห่งชาติ
๑๒๐ ชั้น ๗ อาคารซี ศูนย์ราชการเฉลิมพระเกียรติ ๘๐ พรรษา ๕ ธันวาคม ๒๕๕๐
ถนนแจ้งวัฒนะ แขวงทุ่งสองห้อง เขตหลักสี่ กรุงเทพมหานคร ๑๐๒๑๐ อีเมล saraban@ncsa.or.th

สารบัญ

สารบัญ 2
บทสรุปสำหรับผู้บริหาร 3
การวิเคราะห์เชิงเทคนิค 4
Technique Tactics and Procedures (MITRE ATT&CK) 9
Indicator of Compromise (IOCs) 10
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ 11
อ้างอิง (References) 14

บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้เป็นการวิเคราะห์ช่องโหว่ความปลอดภัยแบบ Remote Code Execution จำนวนสองช่องโหว่ในแกนหลัก (Core engine) ของภาษา PHP ที่มีรหัส CVE-2026-109618 และ CVE-2026-109619 โดยทั้งสองรายการได้รับคะแนน CVSS v4.0 เท่ากันที่ 9.2 (Critical) ช่องโหว่กลุ่มนี้ถูกตั้งชื่อโดยนักวิจัยว่า “PHP2Shell”

ช่องโหว่นี้อยู่ในตัวภาษา PHP เอง ไม่ได้อยู่ในโค้ดของเว็บไซต์ เว็บไซต์ที่เขียนโค้ดอย่างระมัดระวังแล้วจึงยังได้รับผลกระทบ ปัญหาเกิดขึ้นเมื่อ PHP ต้องหยุดประมวลผลคำขอกลางคันเพราะหน่วยความจำไม่พอหรือเกิดข้อผิดพลาดร้ายแรง ซึ่งทำให้ข้อมูลในหน่วยความจำค้างอยู่ในสภาพที่ไม่เรียบร้อย เมื่อคำขอถัดไปเข้ามาใช้ส่วนประมวลผลเดิม ระบบอาจหยิบข้อมูลที่ค้างอยู่นั้นมาใช้ต่อ ผู้โจมตีอาจอาศัยจุดนี้สั่งรันคำสั่งบนเซิร์ฟเวอร์ได้โดยไม่ต้องล็อกอิน แอบอ่านข้อมูลในหน่วยความจำของเซิร์ฟเวอร์ หรือทำให้บริการหยุดทำงาน

เซิร์ฟเวอร์ทุกเครื่องที่ใช้ PHP ให้บริการเว็บได้รับผลกระทบ ไม่ว่าจะติดตั้งแบบใด (mod_php, PHP-FPM, php-cgi, FrankenPHP, LiteSpeed) โดยทาง PHP ประเมินความเสี่ยงสูงสุดที่ mod_php บน Apache ซึ่งมีการพิสูจน์แล้วว่าคำขอที่ไม่ต้องยืนยันตัวตนสามารถกระตุ้นการโจมตีได้ก่อนที่โค้ดของแอปพลิเคชันจะเริ่มทำงาน ทีม PHP ได้ออกแพตช์แก้ไขเมื่อวันที่ 21 ตุลาคม 2569 ในเวอร์ชัน 8.5.12, 8.4.27, 8.3.36 และ 8.2.35 และย้ำว่าการอัปเดตเป็นมาตรการแก้ไขที่สมบูรณ์เพียงอย่างเดียว ไม่มี Workaround ใดที่ป้องกันได้ครบถ้วน **องค์กรหรือหน่วยงานที่ใช้ PHP จึงควรเร่งอัปเดตโดยทันที** โดยให้ความสำคัญกับระบบที่เปิดสู่อินเทอร์เน็ตก่อนเป็นลำดับแรก

การวิเคราะห์เชิงเทคนิค

กลุ่มเป้าหมายที่มีความเสี่ยง

- เว็บไซต์หรือระบบทุกประเภทที่ให้บริการเว็บด้วย PHP โดยไม่ขึ้นกับว่าใช้ CMS หรือเฟรมเวิร์กใด
- การติดตั้งในรูปแบบ mod_php บน Apache (ความเสี่ยงสูงสุด เนื่องจากมีการพิสูจน์ exploit แบบไม่ต้องยืนยันตัวตนที่ทำงานก่อนโค้ดแอปพลิเคชัน) รวมถึง PHP-FPM, php-cgi, FrankenPHP, LiteSpeed/LSAPI
- CMS และแพลตฟอร์มยอดนิยมที่รันบน PHP เช่น WordPress, Drupal, Joomla เป็นต้น ซึ่งนักวิจัยได้สาธิต Pre-auth RCE ทั้ง 3 รายการแล้ว

เวอร์ชันที่ได้รับผลกระทบ

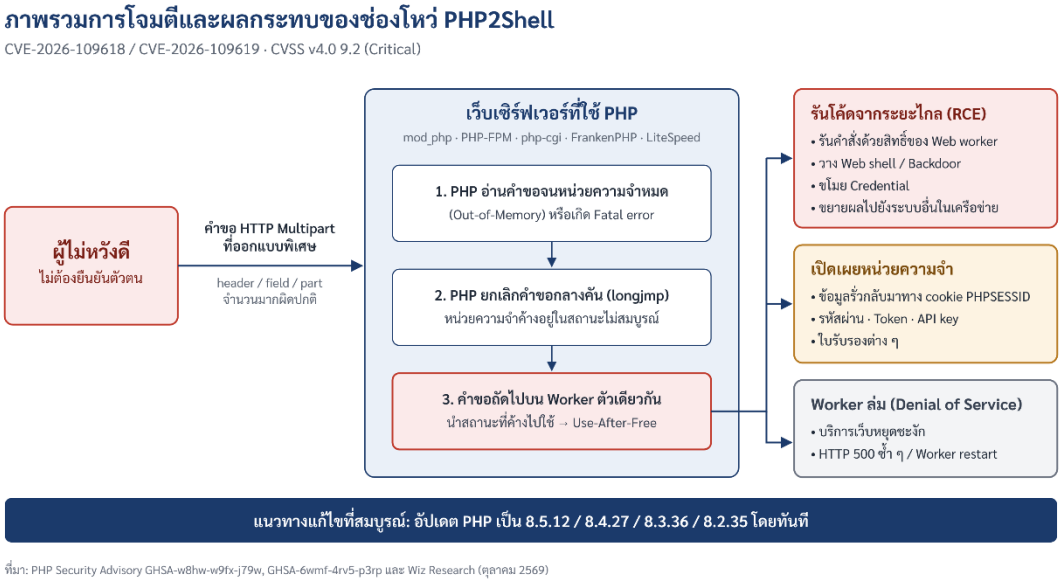
- PHP ทุกเวอร์ชันที่เก่ากว่า 8.2.35, 8.3.36, 8.4.27 และ 8.5.12
- Wiz Research ร่วมกับ Calif และ OpenAI ระบุว่าโค้ดที่มีช่องโหว่ย้อนไปถึง PHP 5.4 (ราวปี 2555) แต่ทีม PHP ไม่ได้ทดสอบยืนยันทุก Release
- Branch ที่หมดอายุการสนับสนุน (8.1, 8.0, 7.x และเก่ากว่า) จะไม่ได้รับแพตช์จากทีม PHP โดยตรง

เวอร์ชันที่ได้รับการแก้ไขแล้ว

- PHP 8.5.12, 8.4.27, 8.3.36 และ 8.2.35 (ออกเมื่อ 21 ตุลาคม 2569 เวลา 14:00 UTC)
- PHP ได้จัดเตรียม source-backport แบบขั้นต่ำสำหรับ PHP 7.4, 8.0 และ 8.1 เพื่อช่วยผู้ที่ build จาก source เอง แม้ Branch จะหมดอายุการสนับสนุนแล้ว
- หมายเหตุ: PHP 8.2 จะหมดอายุการสนับสนุน (End of Life) ในวันที่ 31 ธันวาคม 2569

ผลกระทบที่เกิดขึ้น

ผู้ไม่หวังดีสามารถรันโค้ดบนเซิร์ฟเวอร์ของเหยื่อได้โดยไม่ต้องยืนยันตัวตน ภายใต้สิทธิ์ของ Web worker ซึ่งอาจนำไปสู่การรันคำสั่งระบบปฏิบัติการ การวาง Web shell/Backdoor การขโมย Credential การขยายผลการโจมตีไปยังระบบอื่นในเครือข่าย นอกจากนี้ยังสามารถทำให้เกิดการเปิดเผยหน่วยความจำของ Worker (ซึ่งอาจมีรหัสผ่าน Token API key และใบรับรองต่าง ๆ) และการทำให้ worker ล่ม (Denial of Service)



[ภาพที่ 1 แผนภาพการโจมตีและผลกระทบของช่องโหว่]

รายละเอียดช่องโหว่แต่ละรายการ

CVE-2026-109618 ช่องโหว่ RCE ผ่าน Out-of-Memory (OOM) corruption (GHSA-w8hw-w9fx-j79w)

เมื่อ PHP ประมวลผลคำขอจนหน่วยความจำไม่พอ (Out-of-Memory) PHP จะใช้คำสั่ง longjmp() เพื่อ "กระโดดออก" จากงานที่ทำอยู่ทันทีและยกเลิกคำขอนั้น ปัญหาคือหากหน่วยความจำหมดในจังหวะที่ PHP ยังเตรียมข้อมูลของคำขอไม่เสร็จ การกระโดดออกกะทันหันจะทิ้งข้อมูลที่เตรียมค้างไว้ครึ่ง ๆ กลาง ๆ จากนั้นเมื่อ PHP เข้าสู่ขั้นตอนปิดคำขอ (Shutdown) มันจะเข้าใจว่าข้อมูลที่ค้างอยู่นั้นสมบูรณ์ และนำไปอ่านหรือใช้ตัดสินใจทำงานต่อ ซึ่งผู้ไม่หวังดีสามารถควบคุมข้อมูลตรงจุดนี้ได้

โดยจุดสำคัญอยู่ที่พฤติกรรมของแต่ละวิธีติดตั้ง PHP ดังนี้

- **Apache mod_php (เสี่ยงที่สุด):** ไม่ว่าการเริ่มต้นคำขอจะสำเร็จหรือไม่ mod_php จะรันขั้นตอนปิดคำขอ (Shutdown) เสมอ ผู้ไม่หวังดีจึงเพียงส่งคำขอแบบ Multipart (รูปแบบเดียวกับการอัปโหลดไฟล์) ที่ออกแบบให้ PHP ใช้หน่วยความจำจนหมดตั้งแต่ตอนอ่านข้อมูลขาเข้า จากนั้นขั้นตอน Shutdown ก็จะไปใช้ข้อมูลที่ค้างอยู่ครึ่ง ๆ กลาง ๆ นั้น ทั้งหมดนี้เกิดขึ้นก่อนที่โค้ดของเว็บไซต์จะได้ทำงานแม้แต่บรรทัดเดียว
- **PHP-FPM (เสี่ยงน้อยกว่า):** หากการเริ่มต้นคำขอล้มเหลว FPM จะปิด Process ลูกตัวนั้นทิ้งทันที จึงไม่เข้าสู่ขั้นตอน Shutdown และปิดเส้นทางโจมตีข้างต้นได้

อย่างไรก็ตาม จากการวิเคราะห์ซอร์สโค้ด FPM ยังอาจถูกโจมตีได้ในบางเงื่อนไข เพียงแต่ขณะนี้ยังไม่มีผู้สาธิต exploit ที่ใช้งานได้จริงบน FPM สำหรับช่องโหว่นี้

- **FrankenPHP:** ทำงานคล้าย mod_php คือรัน Shutdown เสมอ จึงมีแนวโน้มถูกโจมตีได้เช่นเดียวกัน

CVE-2026-109619 ช่องโหว่ RCE ผ่าน stale global state (GHSA-6wmf-4rv5-p3rp)

เนื่องจาก PHP มี Global state หลายชนิด (error, session, object, extension) ที่ Worker เก็บไว้แต่ชี้ไปยังหน่วยความจำของคำขอหนึ่ง สถานะเหล่านี้ต้องถูก Reset ก่อนที่หน่วยความจำของคำขอจะถูกปล่อย การประมวลผลบางอย่างด้าน Input, Configuration, Session และ Error ทำงานก่อนการเริ่มต้น (Initialization) ปกติของคำขอถัดไป และเผลอเข้าถึง Stale state ที่ยังไม่ถูก Reset ก่อให้เกิด Use-After-Free และหากผู้ไม่หวังดีจัดวาง Heap ได้แม่นยำ ก็สามารถทำให้เกิด RCE ได้

การโจมตีต้องมีทั้ง “Producer” (Operation ที่เก็บ State ของคำขอหนึ่งไว้ในตำแหน่งที่เข้าถึงได้ภายหลัง) และ “Consumer” (Operation ในคำขอถัดไปที่อ่าน /free/branch บน Stale state นั้น) โดยคำขอทั้งสองต้องไปถึง Worker ตัวเดียวกัน เส้นทางที่ได้รับการยืนยันแล้วมีสองกรณี ได้แก่

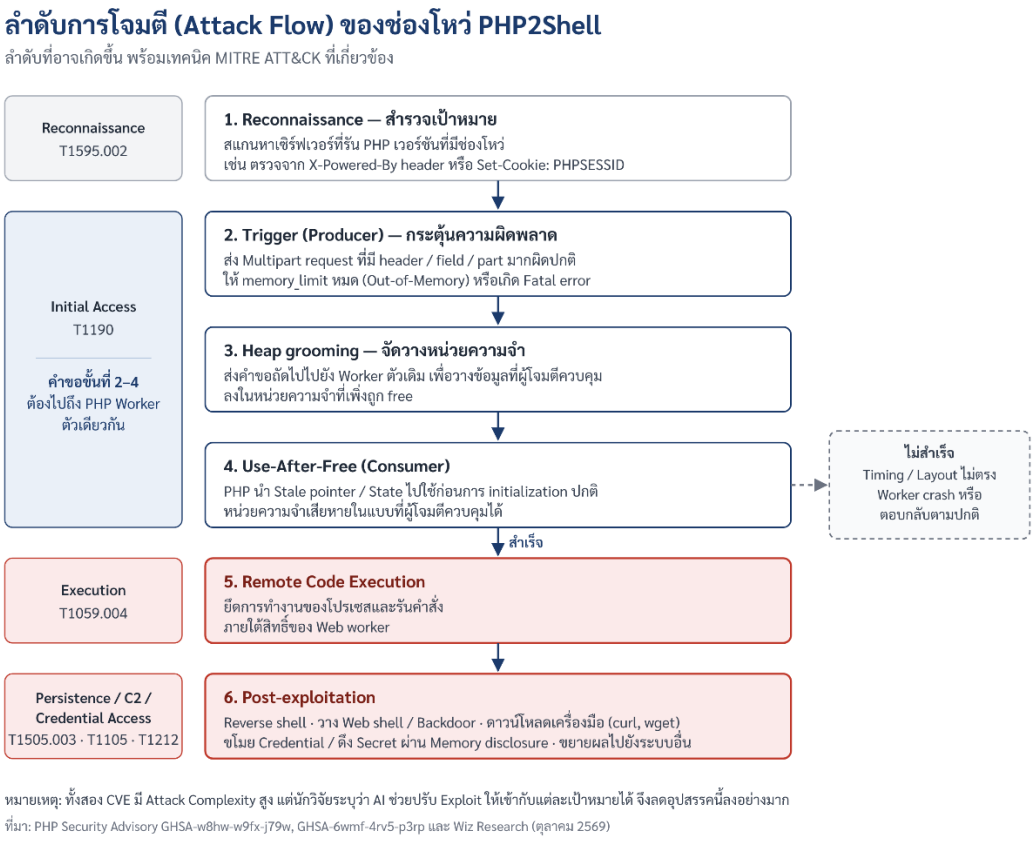
1. **Fatal-error backtrace (PHP 8.5)**: เมื่อเปิด fatal_error_backtraces การรายงาน Fatal error จะเก็บ Backtrace state ของคำขอไว้ และการ Reset ถูกข้ามเสมอระหว่าง Shutdown นี่คือกรณีที่แข็งแรงที่สุดและเป็นพื้นฐานของการยืนยัน RCE บน mod_php
2. **Session upload progress**: ฟีเจอร์นี้เข้าถึง Session handling ก่อนการ Initialization ปกติ จึงสามารถ Consume session/decoder state ที่ค้างจากคำขอที่ถูกขัดจังหวะได้ ฟีเจอร์นี้เปิดใช้งานโดยค่าเริ่มต้นและไม่ต้องให้แอปพลิเคชันเรียก session_start() session lifecycle RCE ได้รับการยืนยันทั้งบน mod_php และ FPM

นอกจากนี้ Wiz Research ยังรายงานเส้นทางที่สามคือ **OPcache type confusion** ซึ่ง Wiz จัดไว้ภายใต้ CVE-2026-109619 เช่นกัน หากการจองหน่วยความจำล้มเหลวระหว่างที่ OPcache บันทึกโค้ดที่คอมไพล์แล้วลงหน่วยความจำร่วม (Shared memory) Worker ตัวอื่นที่ใช้ตำแหน่งเดียวกันต่อจะตีความข้อมูลเป็นชนิดที่ผิด ทำให้สถานะ OPcache ที่ทุก Worker ใช้ร่วมกันเสียหาย Wiz ใช้จุดนี้สาธิตการรันโค้ดจากระยะไกลบน WordPress โดยไม่ต้องยืนยันตัวตน (Pre-auth RCE) ได้สำเร็จ และระบุว่าแพตช์ของ PHP แก้ไขช่องโหว่ทุกรูปแบบนี้แล้ว

ลำดับการโจมตี (Attack Flow) ที่อาจจะเกิดขึ้น

1. **Reconnaissance:** ผู้ไม่หวังดีสแกนหาเว็บไซต์/เซิร์ฟเวอร์ที่รัน PHP เวอร์ชันที่มีช่องโหว่ (เช่น ตรวจจาก X-Powered-By header หรือ Set-Cookie: PHPSESSID)
2. **Trigger (Producer)**: ทำการส่งคำขอ HTTP ที่ออกแบบมาเพื่อกระตุ้นให้เกิด OOM หรือ Fatal error ในจังหวะที่ State ของคำขอยังคงอยู่ มักใช้ Multipart request ที่มีจำนวน header/field/part มากผิดปกติ หรือ body ที่ผ่านการตรวจ post_max_size แต่ทำให้ memory_limit หมด (Exploit body ของ CVE-2026-109619 มีขนาดเพียงประมาณ 450 bytes จึงหลบ WAF rule ตามขนาดได้)
3. **Heap grooming:** ทำการส่งคำขอถัดไปไปยัง Worker ตัวเดียวกัน เพื่อวางข้อมูลที่ผู้ไม่หวังดีควบคุมลงในหน่วยความจำที่เพิ่งถูก free
4. **Use-After-Free (Consumer)**: PHP นำ Stale pointer/State ไปใช้งานในคำขอถัดไปก่อนการ initialization ปกติ ก่อให้เกิดการเสียหายของหน่วยความจำที่ควบคุมได้
5. **Remote Code Execution**: การควบคุมหน่วยความจำนำไปสู่การ Hijack การทำงานและรันคำสั่งภายใต้สิทธิ์ของ Web worker
6. **Post-exploitation**: สร้าง Reverse shell, วาง Backdoor/Web shell, ขโมย Credential, ขยายผลไปยังระบบอื่น หรือใช้ Memory disclosure ดึง Secret ออกมา

หมายเหตุด้านความน่าเชื่อถือ: ทั้งสอง CVE มี Attack Complexity สูง การโจมตีต้องให้คำขอ Producer และ Consumer ไปถึง Worker ตัวเดียวกันและวางข้อมูลได้แม่นยำ หาก Timing หรือ Layout ไม่ตรง Worker อาจเพียง Crash หรือตอบกลับตามปกติแทนการรันคำสั่ง จำนวน Worker ที่มากและการ Recycle จะลดอัตราความสำเร็จต่อครั้ง แต่ไม่ได้กำจัดช่องโหว่ นักวิจัยระบุว่า AI model มีบทบาทสำคัญในการปรับ Exploit ให้เข้ากับแต่ละเป้าหมาย ซึ่งลดอุปสรรคด้านความซับซ้อนนี้ลงอย่างมาก



[ภาพที่ 2 ลำดับการโจมตี (Attack Flow) ของช่องโหว่ PHP2Shell]

Technique Tactics and Procedures

(MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่างๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name
Reconnaissance	T1595.002	Vulnerability Scanning
Initial Access	T1190	Exploit Public-Facing Application
Execution	T1059.004	Unix Shell
Persistence	T1505.003	Web Shell
Command and Control	T1105	Ingress Tool Transfer
Credential Access	T1212	Exploitation for Credential Access

Indicator of Compromise (IOCs)

เนื่องจากช่องโหว่นี้เป็นข้อบกพร่องในตัว PHP runtime เอง การตรวจจับจึงไม่ควรยึดกับ Payload signature ตายตัว (ผู้ไม่หวังดีเปลี่ยนรูปแบบได้ง่าย และ exploit body มีขนาดเล็ก) แต่ควรใช้การตรวจจับเชิงพฤติกรรมสองชั้น ดังนี้

ชั้นที่ 1 สัญญาณระดับคำขอ (Request-level):

- คำขอ Multipart ที่ผิดปกติ ตามด้วย memory-limit error หรือ HTTP 500 ซ้ำๆ
- cookie ตอบกลับ PHPSESSID (Set-Cookie) ที่มีขนาดใหญ่หรือผิดรูปผิดปกติ (สัญญาณของ memory disclosure)
- การเกิด Worker ล่มโดยไม่มีสาเหตุชัดเจน
- คำขอ multipart ที่มีจำนวน Field, Part หรือ Header สูงผิดปกติ

ชั้นที่ 2 พฤติกรรมหลังการโจมตี (post-exploitation):

- กระบวนการ php-fpm, php-cgi, httpd หรือ apache2 spawn shell, script interpreter หรือเครื่องมือดาวน์โหลดเช่น curl, wget
- ไฟล์ใหม่ใน Web root (web shell), cron job หรือ Scheduled task ใหม่, บัญชีผู้ใช้ที่ไม่เคยพบ, การเชื่อมต่อขาออกที่ผิดปกติ

(หมายเหตุ: ณ เวลาจัดทำรายงาน ยังไม่มี IoC แบบ domain/hash ที่ยืนยันจากการโจมตีจริงในวงกว้าง ควรเน้น detection rule เชิงพฤติกรรมและปรับปรุงเมื่อได้ตัวอย่างจริง)

แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

การเฝ้าระวัง (Monitoring)

- สำรวจเซิร์ฟเวอร์ทุกเครื่องที่ใช้ PHP ให้บริการเว็บ ครอบคลุมทุกรูปแบบการติดตั้ง (mod_php, PHP-FPM, php-cgi, FrankenPHP, LiteSpeed) แล้วตรวจสอบเวอร์ชันเทียบกับเวอร์ชันที่แก้ไขแล้ว (8.5.12 / 8.4.27 / 8.3.36 / 8.2.35) กรณีติดตั้ง PHP ผ่าน Linux distribution หรือผู้ให้บริการ LTS ให้ตรวจสอบประกาศแพตช์ของผู้ให้บริการรายนั้น เนื่องจากเลขเวอร์ชันอาจไม่ตรงกับของ PHP โดยตรง
- เปิด log_errors=On พร้อม error_log ที่ใช้งานได้ และ error_reporting=E_ALL ในช่วงทำการ audit เพื่อตรวจหา Warning/Deprecation/Fatal error ที่เกิดซ้ำก่อนสคริปต์เป้าหมายทำงาน รวมถึง Maximum execution time exceeded
- เฝ้าระวังกระบวนการ PHP worker ที่ spawn child process ที่ไม่คาดคิด และสัญญาณ IOC ทั้งสองชั้นข้างต้น

การป้องกัน (Prevention)

- **อัปเดต PHP เป็นเวอร์ชันที่แก้ไขแล้ว (8.5.12 / 8.4.27 / 8.3.36 / 8.2.35) โดยทันที เป็นมาตรการเดียวที่สมบูรณ์** ให้ความสำคัญกับระบบที่เปิดสู่อินเทอร์เน็ตก่อน แล้ว Restart PHP-FPM หรือ Web server เพื่อให้ build ใหม่โหลด
- ตรวจสอบแพตช์ Backport จาก Linux distribution และผู้ให้บริการ hosting
- แยกระบบที่ patch ไม่ได้ (EOL/unpatched) ออกจากเครือข่ายสาธารณะ หรือวางไว้หลัง VPN/WAF จนกว่าจะแก้ไขได้
- **มาตรการลดความเสี่ยงชั่วคราวหาก patch ไม่ได้ทันที** ด้านล่างลดความเสี่ยงได้เฉพาะบางเส้นทางการโจมตี ไม่ได้ทำให้ระบบปลอดภัย และใช้แทนการอัปเดตไม่ได้ ให้ตั้งค่าในระดับระบบ (php.ini, FPM pool หรือ Apache config) ห้ามตั้งผ่าน ini_set() หรือ .user.ini เพราะจะไม่มีผล และทุกครั้งที่เปลี่ยนค่าต้อง Restart Worker ทุกตัว
    - CVE-2026-109618
        - ตั้ง memory_limit=-1 เช่น ผ่าน php_admin_value ใน FPM pool หรือ Apache config แล้วตรวจสอบว่าไม่มีการตั้งค่าใดใน pool, directory หรือแอปพลิเคชันที่ใส่ค่าจำกัดหน่วยความจำกลับเข้ามา
        - ตั้ง post_max_size ให้ต่ำ (8M หากไม่มีการอัปโหลดไฟล์ หรือ 32M หากมี) และให้ upload_max_filesize ต่ำกว่าค่านี้

        - ลด max_input_nesting_level และจำกัดจำนวน Input variable, Multipart part, Header และไฟล์อัปโหลด รวมถึงจำกัดขนาดคำขอที่ระดับ Web server หรือ Proxy
        - ห้ามใช้ RLIMIT_AS หรือ RLIMIT_DATA แทน เพราะจะทำให้ mmap() ล้มเหลวและกระตุ้นเส้นทางที่มีช่องโหว่โดยตรง หากต้องการจำกัดหน่วยความจำ ให้ใช้ Container หรือ cgroup แทน โดยกำหนดให้ปิดเฉพาะ Worker ที่มีปัญหา
        - ข้อควรระวัง: memory_limit=-1 คือการไม่จำกัดหน่วยความจำเลย ถ้าไม่ได้จำกัดขนาดคำขอหรือใช้ cgroup ควบคู่ด้วย เซิร์ฟเวอร์อาจใช้หน่วยความจำจนหมดเครื่องได้
    - CVE-2026-109619
        - ตั้ง session.upload_progress.enabled=0 เพื่อปิดเส้นทาง Session ที่ยืนยันแล้วว่าโจมตีได้ ฟีเจอร์นี้แทบไม่มีการใช้งาน จึงแนะนำให้ปิดต่อไปแม้อัปเดตแล้ว หากไม่จำเป็นต้องใช้
        - PHP 8.5: ตั้ง fatal_error_backtraces=0 เพื่อปิดเส้นทาง Fatal backtrace ที่ใช้ยืนยัน RCE บน mod_php
        - แก้ไข Fatal error และ Execution timeout (Maximum execution time exceeded) ที่ผู้ไม่หวังดีกระตุ้นได้ เช่น ในขั้นตอนอัปโหลด นำเข้าข้อมูล หรือหน้า Error
        - หากไม่ติดเรื่องประสิทธิภาพ พิจารณาตั้ง pm.max_requests=1 (PHP-FPM) เพื่อให้ Worker แต่ละตัวรับคำขอเพียงครั้งเดียว ค่านี้ลดโอกาสโจมตีสำเร็จเท่านั้น ไม่ได้ปิดช่องโหว่
- พิจารณาใช้ WAF ตรวจจับ Multipart payload ที่ผิดปกติ, Session field ที่ซ้ำผิดปกติ และ Response header ที่ผิดปกติ (เป็น Compensating control ไม่ใช่การแก้ที่ต้นเหตุ) พร้อมทั้งเปิดใช้งาน PHP2Shell Rule
- กำหนด Least privilege ให้กระบวนการ PHP เข้าถึงไฟล์/Credential/บริการที่เชื่อมต่อให้น้อยที่สุด

การรับมือ (Response)

- แยกอุปกรณ์ที่ติดเชื้อ: หากพบการโจมตีและบุกรุกให้ ตัดการเชื่อมต่อออกจากเครือข่ายเพื่อป้องกันการฝัง Backdoor/Reverse shell หรือการส่งออกข้อมูล
- เก็บหลักฐานทางดิจิทัล: รักษาสภาพของเครื่องที่ติดเชื้อไว้เพื่อการวิเคราะห์หาสาเหตุและสืบสวน (Digital Forensics) โดยเก็บ Web server log และ PHP log ไว้เพื่อการวิเคราะห์
- ตรวจสอบขอบเขตการโจมตี: ค้นหา Web shell, ไฟล์ใหม่ใน Web root, Cron/Scheduled task ใหม่, บัญชีที่ไม่คาดคิด และ Child process ที่ผิดปกติ ประเมินทั้งความเป็นไปได้ของ Code execution และ Data disclosure
- ยกเลิกและเปลี่ยนข้อมูล Credentials: เปลี่ยนรหัสผ่านฐานข้อมูล, API key, Session secret และใบรับรองที่ PHP Worker ใช้ และยกเลิก Session ทั้งหมด เนื่องจากข้อมูลเหล่านี้อาจรั่วไหลจากหน่วยความจำ
- กู้คืนระบบ: กู้คืนระบบจาก Backup ที่ปลอดภัยหลังจากได้รับการยืนยันว่าได้ทำการอัปเดต PHP เป็นเวอร์ชันที่ได้รับการแก้ไขช่องโหว่เรียบร้อยแล้ว
- รายงานเหตุการณ์: แจ้งเหตุการณ์ไปยังหน่วยงานที่เกี่ยวข้องภายในองค์กรและหน่วยงานด้านความมั่นคงปลอดภัยไซเบอร์

อ้างอิง (References)

1. PHP.net “Security advisory: Critical remote code execution vulnerabilities, update now (21 Oct 2026)”
   https://www.php.net/archive/2026.php#2026-10-21-5
2. PHP “Security Advisory GHSA-w8hw-w9fx-j79w (CVE-2026-109618, RCE via OOM corruption)”
   https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w
3. PHP “Security Advisory GHSA-6wmf-4rv5-p3rp (CVE-2026-109619, RCE via stale global state)”
   https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp
4. CVE.org “CVE Record CVE-2026-109618”
   https://www.cve.org/CVERecord?id=CVE-2026-109618
5. CVE.org “CVE Record CVE-2026-109619”
   https://www.cve.org/CVERecord?id=CVE-2026-109619
6. Wiz Research “PHP2Shell: AI uncovers critical RCE vulnerabilities affecting 70% of the Internet”
   https://www.wiz.io/blog/php2shell-php-rce
7. Imperva Threat Research “PHP2Shell: When PHP Runs Out of Memory at the Wrong Moment”
   https://www.imperva.com/blog/php2shell-vulnerability-updates/
8. SOCRadar “PHP2Shell: CVE-2026-109618 & CVE-2026-109619”
   https://socradar.io/blog/php2shell-cve-2026-109618-cve-2026-109619/
9. Abstract Security “PHP2Shell: Critical PHP RCE Vulnerabilities CVE-2026-109618 and CVE-2026-109619”
   https://www.abstract.security/blog/php2shell-critical-php-rce-vulnerabilities-cve-2026-109618-and-cve-2026-109619

แบบประเมินรายงานข่าวกรองไซเบอร์

https://forms.gle/naPrKU5ruwGu66GC6

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน
ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ
โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th